

# Trust But Verify: Analyzing the Tradeoffs of AI-Generated Code

Aman Swaraj<sup>1</sup>, Indian Institute of Technology Roorkee

Sandeep Kumar<sup>2</sup>, Indian Institute of Technology Roorkee

*// AI tools have become extensively integrated into software engineering today. Still, problems have remained regarding the quality and reliability of their answers. In this study, we have conducted a two-dimensional analysis for understanding the abilities and limitations of these tools. //*



**SINCE THE LAUNCH** of ChatGPT, many claims have been made about AI tools changing the programming practices of developers.<sup>1,2</sup> However, despite their growing adoption, concerns have remained

regarding the quality and security of their responses, especially in coding-related tasks.<sup>3,4</sup>

Given these concerns, it has now become essential to study the role of AI in developer productivity through evidence-based analysis. In this connection, evaluating AI-generated responses to real-world developer

queries as found on reputed forums like Stack Overflow (SO) can prove really valuable, as such analyses enable us to compare AI and human answers and thereby gain a better understanding of their relative strengths.

Besides, this type of approach to benchmark AI and human solutions has also been supported in the recent SO survey,<sup>a</sup> where 75% of participants agreed that human developers are the ultimate referees of quality and correctness.

In addition to this technical analysis, we believe that studying the experience of programmers who actively engage with these tools can also be quite helpful, as it has been proved that mining developer comments on open source forums can help in finding important trends and challenges within software communities.<sup>5,6</sup>

We can further complement this analysis with targeted surveys involving participants from academia and industry and understand their prevailing preferences and trust in AI tools.

In this way, through our multimethod approach, which combines both quantitative analysis and qualitative perception, we can provide a holistic understanding of AI's strengths and limitations.

Based on these objectives, we phrase the following research questions (RQs):

- **RQ1:** How does the code generated by ChatGPT compare to accepted answers of SO in terms of quality and security?
- **RQ2:** What are the prevailing opinions of developers regarding the use of AI tools in comparison to community forums?

<sup>a</sup><https://survey.stackoverflow.co/2025/ai/#2-ai-and-humans-in-the-future>

- RQ3: How do findings from large developer surveys contribute to our understanding of AI tool adoption and trust?

## Quantitative Analysis Setup

### Technical Data Set Generation

To enable a high-quality analysis, we kept our first condition to select only those SO posts that contained a validated answer and had good community upvotes. Further, we kept an additional condition for the answers to have a minimum of 15 lines of code so that trivial snippets or solely theoretical explanations could be filtered.

Next, to reduce the chances of data contamination, we selected SO questions posted after 1 January 2022. However, despite this filter, some queries may still overlap with the training cutoff of the AI model used in our study. We discuss this risk separately in threats to validity.

Regarding question tags, we focused on two widely used programming languages, Java and Python. Keeping the earlier-mentioned conditions, we obtained a total of 20,000 Python and 5,300 Java questions.

From this population, we took the top 2,000 questions for both languages, sorted by upvotes. To validate our sampling, we calculated the margin of error (MoE) and minimum detectable difference (MDD) using a finite-population correction. The MoE values came around 2.08% for Python, 1.73% for Java at a confidence level of 95% and worst-case variance of  $p = 0.5$ . Similarly, the MDD values came 4.2% for Python and 3.5% for Java at a significance level of 0.05 and power 0.80. Together, these values ensured the statistical significance of our sample. Additionally, to examine the

representativeness of our sample, we ran mean-comparison and Kolmogorov-Smirnov test on key attributes, such as Lines of Code (LOC) and question-score, which also showed no significant differences.

We also computed average LOC for the samples, which came 42.2 for Java and 36.3 for Python, ensuring sufficient depth for code analysis.

Next, for AI responses, we used ChatGPT, the most widely adopted AI tool among developers, as reported for two consecutive years in the SO Developer Survey 2024<sup>b</sup> and 2025.<sup>c</sup>

Following the reasoning of prior work,<sup>7</sup> we also replicated the scenario of how a developer, especially newcomer, would use AI tool, and therefore utilized the web interface of the free model (4o-mini), sending each question as is without any prompt formatting.

The complete data set and the reproducibility scripts can be accessed online.<sup>d</sup>

### Tools for Code Analysis

We utilized static analysis tools in our study owing to their efficiency and objective assessment. Accordingly, for Python, we used two widely adopted tools: Pylint<sup>e</sup> to evaluate coding standards, common errors, and overall code quality and Bandit<sup>f</sup> for detecting security issues, such as hardcoded secrets, insecure functions, and improper handling of sensitive data.

Similarly, for Java, we employed PMD<sup>g</sup> to identify code-quality issues (e.g., unused variables, empty catch

blocks, suboptimal expressions) and Semgrep<sup>h</sup> for pattern-based detection of security vulnerabilities and best practice violations.

Following the static analysis, we conducted a qualitative review of a subset of 100 solutions using a scoring framework (shared in our repository) aligned with the quality and security criteria of our static tools. Finally, after manual validation, we analyzed the issues reported by the static tools to identify recurring patterns.

## Developer Feedback Collection

### Mining Community Discussions

Our next phase involved gathering developer perspectives from well-known forums, such as Reddit and meta discussions of SO.

Since our study focused on SO and ChatGPT, we used specific keywords for searching, such as "ChatGPT versus Stack Overflow," "AI versus Human," "ChatGPT," "AI Code," and "Stack Overflow."

From these initial results, we filtered the threads based on upvotes, comments, and participant reputation. However, we did not keep any specific threshold for these filters, as we observed that while certain threads had lower ratings, they were closely related to our discussion theme.

Subsequently, using this approach, we collected a total of 4,159 comments from 2,081 unique users (2,811 from 21 Reddit threads and 1,348 from 16 SO meta posts).

**Manual Annotation.** Since online discussions can contain noise, so as our next step, we used BERTopic<sup>i</sup> to generate and examine visual

<sup>b</sup><https://survey.stackoverflow.co/2024/technology/#1-ai-search-and-developer-tools>

<sup>c</sup><https://survey.stackoverflow.co/2025/technology/#1-large-language-models>

<sup>d</sup>[https://github.com/AnonymousResearch173/ChatGPT\\_Critical\\_Evaluation](https://github.com/AnonymousResearch173/ChatGPT_Critical_Evaluation)

<sup>e</sup><https://pypi.org/project/pylint/>

<sup>f</sup><https://pypi.org/project/bandit/>

<sup>g</sup><https://pmd.github.io/>

<sup>h</sup><https://github.com/semgrep/semgrep>

<sup>i</sup><https://maartengr.github.io/BERTopic/index.html>

clusters from the gathered comments. Subsequently, we set a threshold of 1,000 for our final stage of manual annotation.

In this connection, we adopted the standard tree-based framework from prior work,<sup>89</sup> and annotated each comment with fundamental categories (positive, negative, or neutral)

### Survey Design

The second part of our community study involved gathering insights from developer surveys. To this end, we first studied the findings of the widely recognized SO Developer Survey,<sup>j</sup> focusing on questions related to AI tool adoption and usage patterns.

ChatGPT-generated solutions exhibited a higher number of issues compared to SO answers, except for PMD

and granular emotions (e.g., relief, joy, caution, frustration). Additionally, we labeled them with details, such as containing use case advice, critique, etc., along with their visible preference for human or AI.

The annotation process was carried out by three undergraduate computer science students, each with over three years of programming experience. At the beginning of the task, the first author properly explained all the annotation guidelines to the students and jointly labeled the first hundred comments with them to ensure they understood the nuances of the task. Following this pilot, each of the remaining 900 comments were independently labeled by all three students. Final labels were assigned using majority voting, and inter-rater agreement was measured using Fleiss' Kappa, which gave a satisfactory score of 0.723. All the labeled comments, along with rest of the threads are shared in our repository.

After this step, we created our own survey, particularly aimed at comparing ChatGPT and SO. We included six questions that focused on three specific aspects, namely, usage frequency, perceived trust, and preference between ChatGPT and SO.

We then created the survey on a Google Form<sup>k</sup> and circulated it using 'convenience sampling' among our university and alumni networks. The responses were collected over a period of 30 days, resulting in a total of 200 participants, comprising 56% students (undergraduate, postgraduate, doctoral), 30% software engineers, 8% data scientists, and 6% faculty researchers.

### Insights From Our Technical Analysis

Figure 1 summarizes the results of our static analyses, while the full

list of error counts, descriptions, and corresponding code snippets is available in our repository. For categorizing Pylint and Bandit issues, we followed the standard Common Weakness Enumeration (CWE) nomenclature.<sup>l</sup>

From Figure 1(a)–(d), we can see that ChatGPT-generated solutions exhibited a higher number of issues compared to SO answers, except for PMD. A similar result was also observed in our manual validation phase, where AI solutions to Python questions showed better code structure, but human solutions had better security compliance. Human solutions also excelled in functional correctness, though in Java code, AI had the edge by margin. The analysis sheet of our manual validation subset has also been shared in our repository.

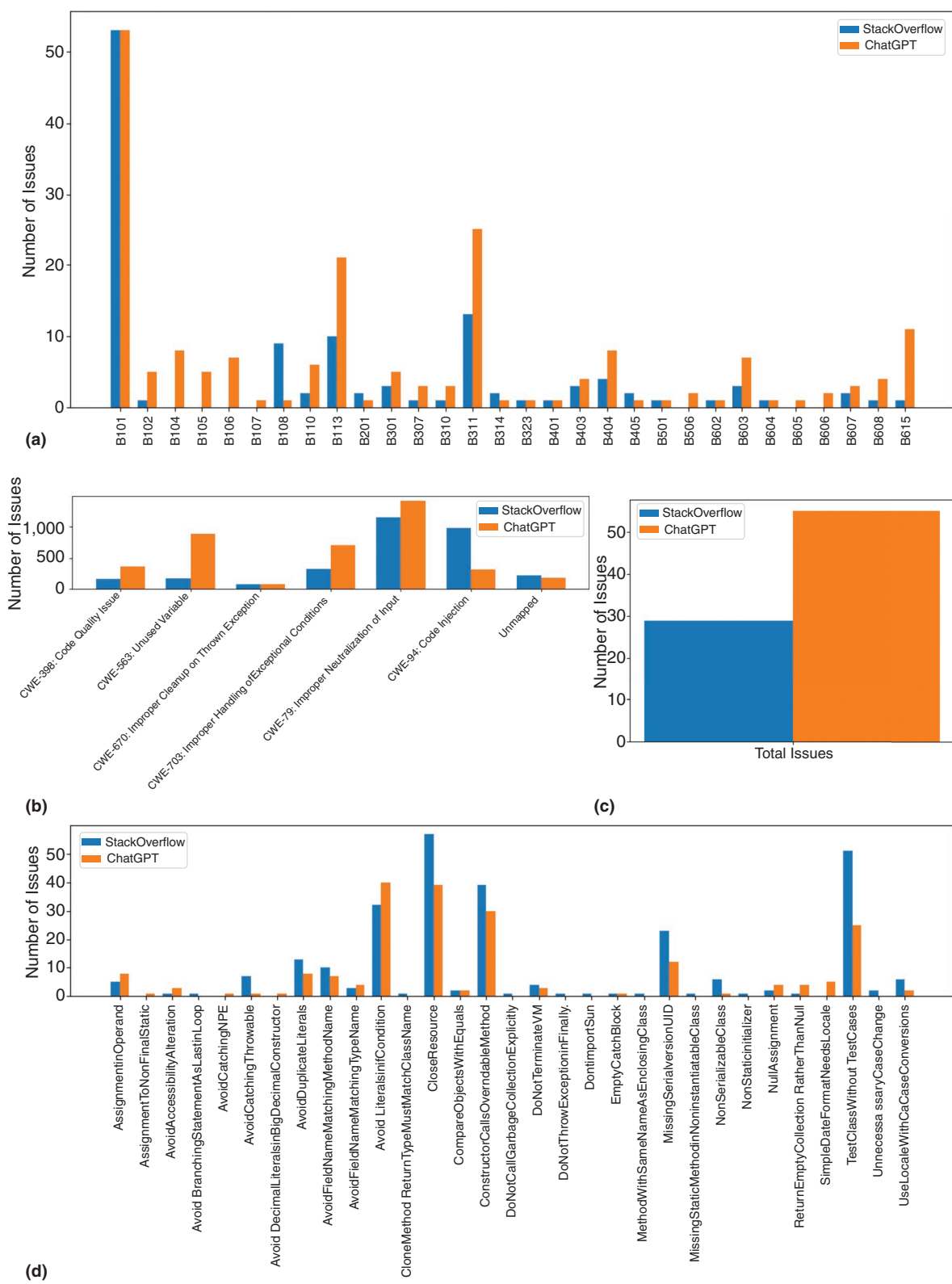
Subsequently, we identified several recurring patterns from our static analysis reports. Next, we summarize some of the key patterns observed:

- *Gap in applying best security practices:* One of the foremost patterns was the recurring presence of security vulnerabilities. For instance, in the Semgrep results, ChatGPT produced 55 issues, with a significant number related to vulnerabilities like "spring-csrf-disabled" and "cookie-missing-secure-flag." Similarly, in results of Bandit, we found that ChatGPT showed a high number "B101" (53 instances) and "B311" (25 instances) issues, which tell about common security flaws like hardcoded secrets and improper file handling. This implies that

<sup>j</sup><https://survey.stackoverflow.co/2025/>

<sup>k</sup>[https://docs.google.com/forms/d/e/1FAIpQLSfI76bwwc4n3Wn5wD7vV-4nVNN2y0p4GncX3T\\_fX5J8Nbhmq/viewform](https://docs.google.com/forms/d/e/1FAIpQLSfI76bwwc4n3Wn5wD7vV-4nVNN2y0p4GncX3T_fX5J8Nbhmq/viewform)

<sup>l</sup><https://cwe.mitre.org/>



**FIGURE 1.** Comparison of code quality and security issues in SO and ChatGPT answers using four static analysis tools, in order: (a) Bandit for Python, (b) Pylint for Python, (c) Semgrep for Java, and (d) PMD for Java.

AI tools sometimes pick simple application programming interface calls without considering the security context.

- *Frequent syntax and import errors:* We also noticed a large number of syntax and import-related errors. Such as in Pylint, we found "E0401" coming 711 times and "E0602" 1425 times. These issues are related to missing imports or references that are not resolved properly. This tells us about the problem of AI agents with complex coding environments and thus, human supervision will be required where correct module imports and references are crucial.
- *Mismanagement of data structures:* Next, we found certain error codes, such as "B607" and "B603" in Bandit, that were related to improper data handling or inefficient use of data structures. Although these are less frequent as compared to security vulnerabilities, these issues suggest that ChatGPT struggles to optimize data structures. Thus, developers need to have a deeper understanding of context and performance tradeoffs while rectifying these issues. Similarly, in the PMD Java analysis, 'TestClassWithoutTestCases' appeared 25 times and 'CloseResource' 39 times in ChatGPT code, indicating frequent gaps in test coverage and resource management.
- *Reliance on deprecated or default libraries:* Some issues, like B405 (insecure XML parsers) and B301 (pickle use), appeared more often in ChatGPT as compared to human solutions. This demonstrates that AI tools are more prone to suggesting legacy

APIs, which in turn indicates a tendency to optimize functional correctness over up-to-date security practices. Similarly, B404 (use of `xml.etree.ElementTree` without secure parsing) also indicates a pattern where GPT-generated examples default to standard library usage without applying security best practices.

- *Inconsistent handling of warnings and errors:* Apart from major security concerns, Pylint results also revealed several instances of inconsistent error handling in ChatGPT's code, such as "W0707" and "W0105," which relate to unnecessary exception handling and logging practices. While these problems may also not necessarily be fatal, they could lead to poor bug diagnostics in long term. Similarly, low-impact warnings, such as unused imports (W0611) or expressions not assigned (W0106), also appeared more frequently in the ChatGPT snippets. These patterns suggest that while AI can produce code suitable for learning, it may not always be production-ready.
- *Structural and maintainability weaknesses:* Finally, we also noted a few patterns of maintainability issues in ChatGPT code. For instance, in Python, Pylint flagged W0621 (redefined outer-scope variable) 722 times in ChatGPT code versus 109 in SO, indicating frequent instances of variable-shadowing. Similarly, in PMD analysis, `AvoidLiteralInIfCondition` (40 occurrences) and `ConstructorCallsOverridableMethod` (30 occurrences) also suggest that ChatGPT often produces code with maintainability or design concerns. This

shows the habit of ChatGPT to give verbose solutions but affecting the efficiency.

## Insights From Developer Experiences

### Community Discussions Mining

Our RQ2 focused on understanding developers' opinions on the usage of AI tools. Here in [Table 1\(A\)](#), we highlight the key patterns and statistics that emerged from the labeled set.

We can see that developer comments reveal a mixed outlook on the use of AI tools. While most developers (38.7%) showed a preference for ChatGPT, others wanted to use SO (21.2%), and others still wanted to utilize both (33.9%). We also noticed some clear positive sentiment coming from users (25.7% of comments) who experienced benefits from AI usage. In contrast, some 22.0% expressed caution or skepticism toward the AI tool. Some developers also provided use-case advice (10.9%) or cases of critical evaluations (22.6%), which very much enriched our data set with practical evidence.

### Survey Findings

After analyzing these comments, we next complemented these findings with survey data. Our primary study involved the SO Developer Survey 2025, focusing on the questions related to AI tool usage. These results are summarized in [Table 1\(B\)](#).

As evident, the results show a mixture of preference and caution. While adoption is highest for tasks like routine coding (30%), basic queries (54%), and learning (33%), critical tasks like deployment (6%) or customized applications, such as robotics (3%), demanded more human attention. Many respondents

**Table 1. Developer preferences, sentiments, and AI usage insights from SO and the 2025 Developer Survey.**

| (A) Summary of Preferences, Sentiments, and Notes from Comments |                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                         |
|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category                                                        | Percentage                                                                                                                                                                                                                                                                                                                                                                                                                | Example comment                                                                                                                                                         |
| Preference for ChatGPT                                          | 38.7%                                                                                                                                                                                                                                                                                                                                                                                                                     | I'm slowly decreasing my reliance on SO and increasingly incorporating ChatGPT in my workflow.                                                                          |
| Preference for SO                                               | 21.2%                                                                                                                                                                                                                                                                                                                                                                                                                     | If people want answers from ChatGPT, they will go to ChatGPT immediately and not to SO. If people come to SO, they're looking for high-quality human-generated answers. |
| Developers who feel the requirement for both                    | 33.9%                                                                                                                                                                                                                                                                                                                                                                                                                     | SO is useful for setup-related issues and for programming issues. I use ChatGPT these days.                                                                             |
| Comments with positive sentiment around AI tool usage           | 25.7%                                                                                                                                                                                                                                                                                                                                                                                                                     | I have strictly been using ChatGPT , and the amount of work I am producing is BEYOND me.                                                                                |
| Negative sentiment expressed towards usage of ChatGPT           | 22.0%                                                                                                                                                                                                                                                                                                                                                                                                                     | I don't need speedy and wrong answers. People come to SO to copy the correct answer.                                                                                    |
| Use case/technical advice given by developers                   | 10.9%                                                                                                                                                                                                                                                                                                                                                                                                                     | Yes, I find ChatGPT more useful to check the programming issue and other alternative ways to do it. For setup-related issues SO is best.                                |
| Critical review of ChatGPT/ caution expressed                   | 22.6%                                                                                                                                                                                                                                                                                                                                                                                                                     | While the solutions often look correct to the untrained eye, they are frequently invalid in various subtle ways.                                                        |
| (B) Key Insights from the SO 2025 Developer Survey              |                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                         |
| Theme                                                           | Key insights                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                         |
| Usage and general sentiment                                     | 79% of developers reported actively using AI tools in their workflows, with 60% expressing a positive overall sentiment.                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                         |
| Trust in AI tools                                               | 33% of the participants showed trust in AI outputs, while 46% expressed skepticism. Only 5% consider AI reliable for complex coding tasks, highlighting the need for human knowledge.                                                                                                                                                                                                                                     |                                                                                                                                                                         |
| Integration with development cycle                              | Developers are comfortable with AI for routine coding (30%), learning new concepts (33%), and solving simple problems (54%), but they avoid important tasks, such as deployment (6.2%). As far as adoption is concerned, it is highest among software engineers (83%) and lowest among robotics professionals (3%). Among tools, ChatGPT (81%) and GitHub Copilot (67%) are the most widely used ones.                    |                                                                                                                                                                         |
| Frustrations and challenges                                     | Main reason for frustration are AI's suggestions that look like "almost correct" but require large corrections (66%) and then debugging AI-generated code (45%). Accuracy concerns (57%), privacy issues (56%), and cost (50%) came up as major barriers for developers. Around 20% of developers also reported decreased confidence in their abilities because of dependence on AI. Only 4% experienced no frustrations. |                                                                                                                                                                         |
| Impact on productivity                                          | 52% of developers reported significant productivity gains, while 42% reported minimal improvements, and 70% noted faster completion of specific tasks. However, only 17% observed enhanced team collaboration by using AI tools.                                                                                                                                                                                          |                                                                                                                                                                         |
| Human-in-the-loop scenarios                                     | 75% of developers still seek human assistance when they lack trust in AI outputs. Over half reported that AI alone is insufficient for learning best practices or acquiring comprehensive knowledge.                                                                                                                                                                                                                      |                                                                                                                                                                         |

(52%) experienced significant productivity gains by using AI, but at the same time, there were also concerns raised over its accuracy (57%) and

contextual understanding of problem statements. This is similar to our earlier findings that developers are choosing to trust but also want to verify.

After collecting these findings from the SO developer survey, we also conducted a focused study to understand comparative preferences

between SO answers and ChatGPT responses. The findings, as summarized in Figure 2(a)–(d), again reveal mixed patterns in tool selection and trust factors. On average, participants reported accessing ChatGPT slightly more often (2.76 mean) than SO (2.68 mean) and rated its usefulness marginally higher (3.72 versus 3.57). ChatGPT also came out as the first choice (62%) as compared to SO (33%), but it edged ChatGPT in terms of confidence (3.12 versus 2.05).

## Key Takeaways and Implications

Based on our results, we summarize the key findings of our research questions and their practical implications.

### Takeaway from RQ1

Across our static-analysis checks and manual review, we observed that ChatGPT solutions had proper structure, but they were more prone to errors, especially in security-related aspects. In contrast, SO answers were sometimes inconsistent in style, but they showed better contextual understanding and fewer critical issues. Based on this, we recommend using AI-generated code for quick routine tasks but relying on human-written or carefully reviewed solutions (in case of AI) when accuracy and security are important.

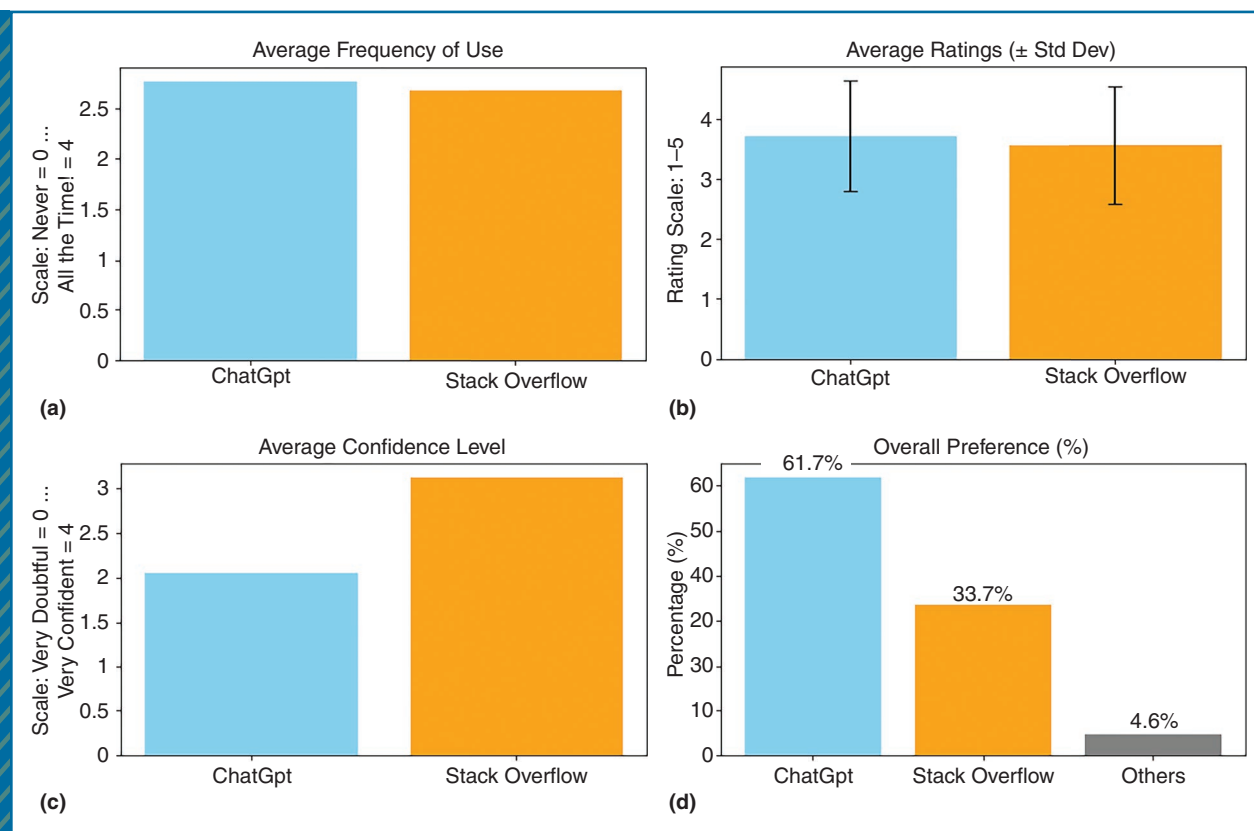
### Takeaway from RQ2

Next, in our community forum mining, we saw how developers appreciated AI

tools for their speed, readability, and professional guidance, but at the same time, they expressed caution, selecting human answers when it came to debugging, searching for precise solutions, or handling edge cases. This suggests that AI can work most effectively when it is being used along with human oversight.

### Takeaway from RQ3

Finally, our survey analysis also yielded similar findings as identified in RQ1 and RQ2. Developers chose AI tools for routine coding, learning, and explanation tasks, but hesitation remained for high-stakes or security-critical work. The survey further revealed that uninformed reliance on



**FIGURE 2.** Findings from our targeted survey showing developer preferences, confidence, and quality between ChatGPT and SO answers. (a) Average frequency of use. (b) Average ratings ( $\pm$  standard deviation). (c) Average confidence level. (d) Overall preference (%).

AI can be counterproductive, as debugging AI-generated code emerged as one of the most common sources of frustration. Our targeted survey also reiterated this pattern, as the participants preferred ChatGPT more in the preference poll, but chose SO for trust.

### Threats to Validity

Our first threat concerns the generalizability of our findings as our data set is limited to 4,000 questions, two programming languages, and analyses based on static tools. We addressed the data set size aspect in our methodology section through appropriate statistical tests. Next, concerning programming languages, Python and Java are among the most widely used, and thus can offer reasonable coverage of developer scenarios. Finally, regarding our reliance on static analysis tools, we acknowledge that these tools are limited in their ability to detect all code issues, but at the same time, they are widely adopted in the software engineering domain as they provide a consistent basis for analyzing the code snippets. To further combat this threat, we conducted a manual analysis of a subset of our data set.

We also acknowledge the limitation of relying only on ChatGPT for our comparative analysis. However, as outlined in the methodology, ChatGPT has been the most widely used AI tool, making it an appropriate baseline for our study. Future work could expand this analysis by incorporating multiple large language models to provide broader coverage.

Our next threat concerns data contamination, as some SO questions in our data set likely overlap with the training cutoff of ChatGPT-4o-mini (October 2023) and thereby benefit the AI. While we

admit this possibility, our study aim was not to particularly find conclusive winners, but to identify the comparative strengths of AI tools and developer knowledge. Further, in our study, we identified several cases where human answers outperformed

research community. In this connection, most existing works were based on the DevGPT data set<sup>m</sup> introduced in the early years of ChatGPT.<sup>10,11,12</sup>

While these studies provided useful insights into the shortcomings of AI-generated code, they were

Developers chose AI tools for routine coding, learning, and explanation tasks, but hesitation remained for high-stakes or security-critical work.

AI, despite the advantage of training cutoff; this point strengthens our analysis.

Another risk to our analysis could be the reliance on the SO accepted answer as the human baseline, which may not be the best solution. While we acknowledge this chance, the threat is mitigated considering the reputation of SO and community upvotes to those questions.

One final concern relates to our thread selection procedure, which involved qualitative judgment and may introduce subjectivity. However, our approach was not purely subjective, as during manual screening, we took note of several important metrics, such as upvotes, comment counts, and participant reputation. As a result, this approach helped us to capture both highly visible discussions and theme-specific ones. Furthermore, all the threads that were selected for analysis have been shared publicly for replicability.

### Related Work

Analyzing AI-generated code has recently gathered attention in the

mainly conducted in isolation and utilized ChatGPT-3.5, which has since been deprecated.

In contrast, few studies focused on comparing AI-generated responses with human solutions. For instance, Jamil et al.<sup>13</sup> evaluated ChatGPT 3.5 and 4 against the Humaneval benchmark and demonstrated the AI's ability to match the best human solutions. While valuable, the Humaneval data set comprises synthetic questions that don't reflect real-world queries as found in SO questions.

In the context of SO versus ChatGPT, Hamer et al.<sup>14</sup> utilized CodeQL to compare the vulnerability of 108 Java snippets and reported that ChatGPT answers showed 20% fewer security vulnerabilities, a trend observed in our study as well (26%).

In another study, Jinrun et al.<sup>15</sup> evaluated SO and ChatGPT on task completion rate, where ChatGPT excelled SO on algorithmic tasks, but underperformed on debugging tasks. However, their evaluation was based

<sup>m</sup><https://github.com/NAIST-SE/DevGPT>



**AMAN SWARAJ** is a Ph.D. student at the Indian Institute of Technology Roorkee, Roorkee-247667, India. His research interests include information retrieval utilizing natural language processing and computer vision techniques as well as human-centered studies in computing. Swaraj received an M.Tech in computer science and engineering from the National Institute of Technology, Delhi. Contact him at aman\_s@cs.iitr.ac.in.



**SANDEEP KUMAR** is a professor in the Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee 247667, India. His research interests include web services, software engineering, and artificial intelligence. Kumar received his Ph.D. in computer science and engineering from the Indian Institute of Technology BHU, Varanasi. He is a Senior Member of IEEE and ACM. Contact him at sandeep.garg@cs.iitr.ac.in.

on only six questions, limiting their generalizability.

The closest work to our study was conducted by Kabir et al.,<sup>7</sup> who analyzed ChatGPT-3.5 answers to 517 SO questions followed by a user interview. Their analysis reported around 56% of ChatGPT answers containing issues, similar to patterns we observed in our study. Additionally, their user study revealed preference for ChatGPT but showed more confidence in SO, another pattern consistent with our findings. However, their evaluation approach solely relied on manual analysis, only analyzed the AI responses. In contrast, our work analyzes both human and AI solutions using objective static tools for consistency.

Another factor that motivated our study was the absence of community feedback from prior works, which is necessary for understanding developer sentiment and preferences.

In this study, we have presented a data-driven analysis of AI-generated solutions to technical questions on community forums. We have combined technical evaluation with community-driven feedback from surveys and discussions and provided a holistic understanding of where AI performs well and where human expertise remains essential. Our findings show that while AI offers accessible solutions, human answers still outperform in contextual accuracy, domain-specific optimization, and handling edge cases. 

#### Acknowledgment

The authors acknowledge the funding received under the Information Security Education and Awareness Phase-III and the Scheme for Promotion of Academic and Research Collaboration schemes of Government of India. We also acknowledge the contributions of Bhoomi Bonal, Manashree

Kalode, and Neha Gujjar for their assistance in preparing the data set used in this study.

#### References

1. J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, "Chat-GPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," in *Generative AI for Effective Software Development*, A. Nguyen-Duc, P. Abrahamsson, and F. Khomh, Eds. Cham, Switzerland: Springer-Verlag, 2024, pp. 71–108.
2. E. Brynjolfsson, D. Li, and L. Raymond, "Generative AI at work," *Quart. J. Econ.*, vol. 140, no. 2, pp. 889–942, 2025, doi: [10.1093/qje/qjae044](https://doi.org/10.1093/qje/qjae044).
3. Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT," *IEEE Trans. Software Eng.*, vol. 50, no. 6, pp. 1548–1584, Jun. 2024, doi: [10.1109/TSE.2024.3392499](https://doi.org/10.1109/TSE.2024.3392499).
4. A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. Jiang, "GitHub copilot AI pair programmer: Asset or liability?" *J. Syst. Softw.*, vol. 203, Sep. 2023, Art. no. 111734.
5. R. Abdalkareem, E. Shihab, and J. Rilling, "What do developers use the crowd for? A study using stack overflow," *IEEE Softw.*, vol. 34, no. 2, pp. 53–60, Mar./Apr. 2017, doi: [10.1109/ms.2017.31](https://doi.org/10.1109/ms.2017.31).
6. A. Barua, S. W. Thomas, and A. E. Hassan, "What are developers talking about? An analysis of topics and trends in stack overflow," *Empirical Softw. Eng.*, vol. 19, no. 3, pp. 619–654, 2014, doi: [10.1007/s10664-012-9231-y](https://doi.org/10.1007/s10664-012-9231-y).
7. S. Kabir, D. N. Udo-Imeh, B. Kou, and T. Zhang, "Is stack overflow obsolete? An empirical study of the characteristics of ChatGPT answers to stack overflow questions," in *Proc. CHI Conf. Human Factors*

- Comput. Syst.*, 2024, pp. 1–17, doi: [10.1145/3613904.3642596](https://doi.org/10.1145/3613904.3642596).
8. N. Novielli, F. Calefato, and F. Lanubile, “A gold standard for emotion annotation in stack overflow,” in *Proc. 15th Int. Conf. Mining Softw. Repositories*, 2018, pp. 14–17.
  9. P. Shaver, J. Schwartz, D. Kirson, and C. O’Connor, “Emotion knowledge: Further exploration of a prototype approach,” *J. Personality Social Psychol.*, vol. 52, no. 6, pp. 1061–1086, 1987, doi: [10.1037/0022-3514.52.6.1061](https://doi.org/10.1037/0022-3514.52.6.1061).
  10. M. L. Siddiq, L. Roney, J. Zhang, and J. C. D. S. Santos, “Quality assessment of ChatGPT generated code and their use by developers,” in *Proc. 21st Int. Conf. Mining Softw. Repositories*, 2024, pp. 152–156.
  11. K. Moratis, T. Diamantopoulos, D. N. Nastos, and A. Symeonidis, “Write me this code: An analysis of ChatGPT quality for producing source code,” in *Proc. 21st Int. Conf. Mining Softw. Repositories*, 2024, pp. 147–151.
  12. M. F. Rabbi et al., “AI writes, we analyze: The ChatGPT python code saga,” in *Proc. 21st Int. Conf. Mining Softw. Repositories*, 2024, pp. 177–181.
  13. M. T. Jamil, S. Abid, and S. Shamil, “Can LLMs generate higher quality code than humans? An empirical study,” in *Proc. IEEE/ACM 22nd Int. Conf. Mining Softw. Repositories (MSR)*, Piscataway, NJ, USA: IEEE Press, 2025, pp. 478–489, doi: [10.1109/MSR66628.2025.00081](https://doi.org/10.1109/MSR66628.2025.00081).
  14. S. Hamer, M. d’Amorim, and L. Williams, “Just another copy and paste? Comparing the security vulnerabilities of ChatGPT generated code and stackoverflow answers,” in *Proc. IEEE Secur. Privacy Workshops (SPW)*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 87–94, doi: [10.1109/SPW63631.2024.00014](https://doi.org/10.1109/SPW63631.2024.00014).
  15. J. Liu, X. Tang, L. Li, P. Chen, and Y. Liu, “ChatGPT vs. stack overflow: An exploratory comparison of programming assistance tools,” in *Proc. IEEE 23rd Int. Conf. Softw. Qual., Rel., Secur. Companion (QRS-C)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 364–373, doi: [10.1109/QRS-C60940.2023.00105](https://doi.org/10.1109/QRS-C60940.2023.00105).



# RECOGNIZE EXCELLENCE

## COMPUTER SCIENCE & ENGINEERING AWARDS

**IEEE Computer Society** honors groundbreaking achievements in research and contributions to the global computing community.

### AWARDS INCLUDE:

- Technical Achievements
- Teaching & Education
- Student Distinction & Scholarships
- Exemplary Service



To learn more, visit  
[computer.org/awards](https://computer.org/awards)



**IEEE  
COMPUTER  
SOCIETY**